Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 12

Lecture – 60

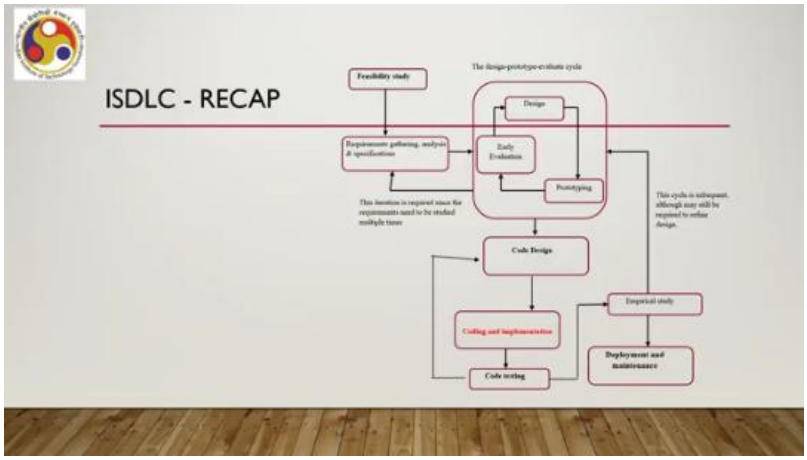
Linking theory to practice

Hello and welcome to the NPTEL MOOC's course on Foundations for Virtual and Augmented Reality Systems. So, we have come to almost the end of this course. Before we conclude, we will cover one more topic, one more interesting topic where we will discuss how to link our theoretical knowledge that we have acquired so far through the previous lectures to practical implementation. So, essentially what we will try to do in this lecture before we conclude is to link the theory to practice.

Now, we have learned so far the idea of XR and the foundational aspects which include several areas including human factors, including interactive system development, concerns including hardware, software, tracking, and many such concepts that we have covered in our discussion so far. But as I said, these are mostly theoretical. So, we have not delved deeper into the practical aspects except the fact that we have discussed one case study to gain some practical knowledge on how to translate the theory to practice.

But a case study is one way to link the theory to practice and it is, of course, not the only way or the best way. It is primarily focused on the broader aspects of development, particularly the interactive system development lifecycle, the stages of the lifecycle, and how those are translated to practice. What are the things involved in translating the theoretical knowledge to practice, primarily focused towards the broader aspects. Now, in this lecture, let us try to understand the linkage between theory and practice from a different viewpoint, which is more towards actual implementation.

How a developer can develop and deploy a system, marrying theory with practice. So, of course, case study gives you some idea. But here we will try to get more implementation level ideas, how the theory is converted to practical ideas. At this stage, it will be pertinent to recollect the interactive system development lifecycle, which we have talked about earlier in details. Comprising of several stages, starting with feasibility study, then requirement gathering and specification.



Then this design prototype evaluate cycle comprising of the design stage, prototyping stage and evaluation stage. This is followed by the code design stage, followed by coding and implementation, followed by testing of the code. followed by empirical study or usability study, followed by deployment and maintenance. Also, there are several loops as we have repeatedly discussed, so I will not go into the details of those. The primary concern of us in this lecture will be this stage, coding and implementation.

So, here we are assuming that We have already completed the design part that means we have gathered requirement, usability requirements, converted those to design ideas by following the iterative design cycle and also we have come up with the broader design of the code or the system. Now, it is the time to translate the code into practice and there we would like to see how our concepts, theoretically acquired concepts can be translated to practice. using available programming environment.

So our focus here will be coding and implementation. So essentially what we will do, we will delve deeper, little deeper on programming environment and system building, which of course is followed by deployment to understand the support available for a developer from the point of view of programming environments. And in order to do that, we will take help of the Unity platform.

It is a very popular and well-known programming environment for developers who wish to work with extended reality systems. Now, the Unity platform comprises the game engine, Unity 3D game engine and a developer's interface, which a developer can use to implement the ideas. So, we will talk about those in this lecture. Before we proceed further, let us quickly try to understand what exactly we are looking for.

So, we are looking for support for implementation of essentially the graphics pipeline that we have discussed earlier, the computer graphics pipeline. including object definition, scene construction, lighting effects, view transformation and of course scan conversion or rendering. Now, we are trying to basically look for support where the developer need not put effort to master the pipeline concepts and implement the pipeline by himself or herself. So, essentially what we are looking for is a support environment where the developer can focus on the immersive experience building rather than implementation of the graphics pipeline. So, with that objective in our mind, let us try to get some basic idea of the Unity developer's platform and then we will proceed on specific developmental support that is

there or that is provided by the platform, the Unity platform. Now, Unity as we have talked about earlier while talking about extended reality software, it is a cross-platform game engine initially released by Unity Technologies in 2005, about 20 years back. It supports both 2D and 3D games or interactive content. and supports over 20 different platforms including PC, mobile, Android based mobile and Apple mobile platforms, iOS.

So, it has a wide support base for different platforms. Now, let us talk briefly about the installation. There is a package called the Unity Hub, which is the primary way to install the Unity Editor or the developer's interface that we are talking about and the Unity Game Engine. So, there are two things. One is the Game Engine, which is a middleware and then interface, which allows developers to build extended reality experience or extended reality systems and deploy it on a specific platform.

So, both can be installed if we are installing Unity Hub. And this is also a way to allow the developer to build and deploy projects. So, through the menu options on the Unity Editor interface, the developers can build and deploy their system. The main advantage of installing Unity Hub is that it supports multiple versions of Unity. So, you can to manage multiple versions of Unity together at one place. For more details about the installation, you can refer to this documentation docs.unity3d.com, their hub under that manual and its index page. You can refer to for different topics on installation. But Unity Hub installation is of course not mandatory. working with Unity, you can separately install the Unity editor and the engine separately also you can install.

In that case, of course, you can install only one and you can deal with only one version of Unity, whereas whereas Unity hub interface allows you to deal with multiple Unity versions, but both ways are possible. Unity Hub essentially is a standalone application that streamlines the way you find, download and manage your Unity projects and installations. It can manually add versions of Unity that are already installed in your system, if any, to the Hub. And it is easy, it makes it easy to install Unity Engine.

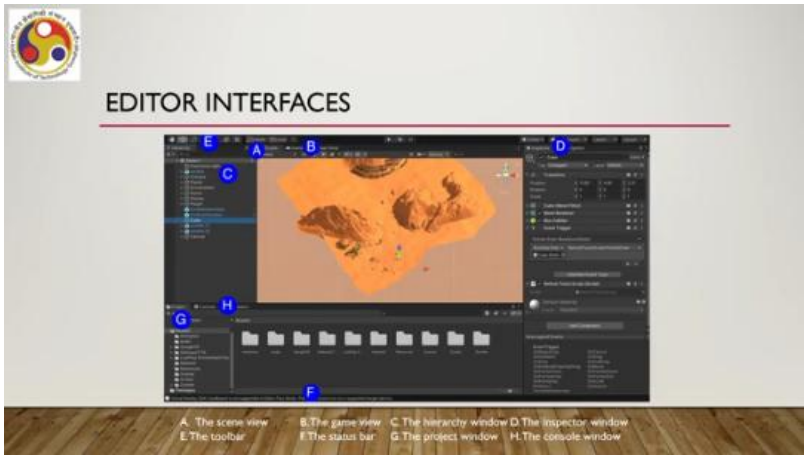
So, it is easy to install Unity Engine using the Unity Hub package. Now, as I said, once you install the package, you get two things. One is the editor interface, other one is the game engine. Game engine gets installed as a middleware and you get editor interface where you can utilize different options to build your system, to develop your system. So, the editor interface allows you to build and deploy realistic immersive environment without having to master and implement the CG pipeline, which was our original objective.

It is essentially an IDE or integrated development environment providing the developer all the features required to build and deploy a system. Now, when you are dealing with extended reality systems through the Unity editor interface, there is a preference tab. In the preference tab on this interface, Unity has XR settings for three types of systems. One is virtual reality, which essentially is as per the editor interface. It simulates a completely different environment around the user, artificial environment.



Then there is this mixed reality setting in which it combines its own environment with the user's real world environment and allows them to interact with each other and augmented reality where it provides layers of content over a digital view of the real world. So, although in our earlier discussion, we said that we will, for the sake of discussion, we will regard, we will consider this augmented reality and mixed reality to be referring to almost the same concept. Here in the Unity interface editor interface, you can actually select settings for each type and then accordingly you will get to do certain things when you choose a particular option.

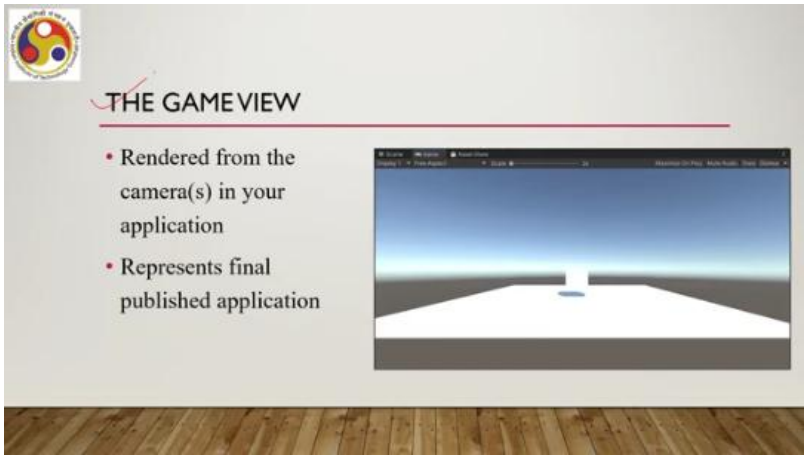
So, this is how the interface looks. It has several components marked as A, B, C, D, E, F, G, H. So, A is basically called the scene view as shown here, then B refers to the game view, C is the hierarchy window, D is the inspector window, E the toolbar, F the status bar, G the project window, and H the console window. So, there are different components; we will not go into all those components here. But to understand the basic thing, we will focus only on these two components, A and B, that is the scene view and the game view, what they refer to.



Other components we will ignore for the sake of this discussion. So let us talk about the scene view. It is an interactive view of the world you are, you meaning the developer, is creating. So essentially, when you start building a scene, then you get to see it in the scene view.

So, in this view, what you can do is play around with the scene. So, you can perform selection, manipulation, and modification of the game objects, which are the fundamental concepts in Unity, in the scene view option. Now, these objects, of course, comprises of the scene. So, these objects put together Creates the scene and you can play around with these objects in the scene view.

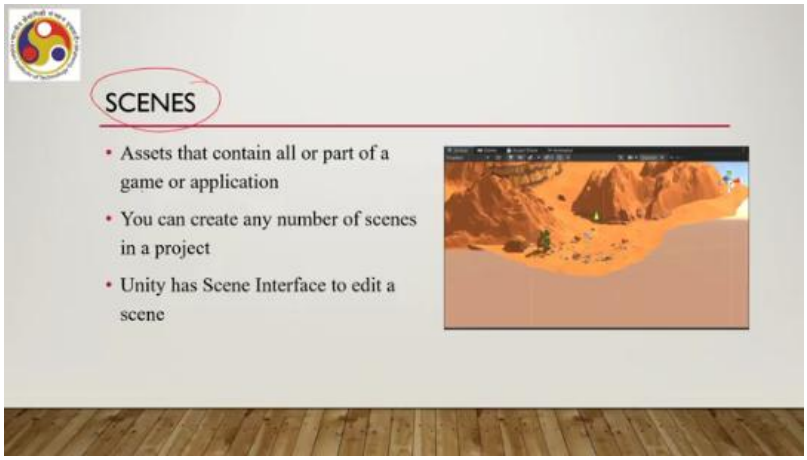
Once the next thing is the game view. So once you are done with your system building, you can or once you think that you have developed a scene, you can view the final scene from the game view. Scene view is the developer's view. Game view is the final view. So, here in GameView, the scene is rendered from the camera set by the developer and it represents the final published application or the deployed application.



So, that is one support you get with Unity. Now, in order to understand what is a scene and how it works, we need to have some basic understanding of certain other things. One such thing is the Unity assets, the concept of asset. Any item that you use in your Unity project to create your app is called an asset. Such assets can represent visual or audio elements in your project such as 3D models or textures, those things.

An asset may come from a file created outside of Unity, so you can import asset from outside or you can build it inside the Unity environment. So, when you are importing, you can import it as a file such as a 3D model, an audio file or an image. Unity also offers creation of internal assets, so both ways are possible. For example, scenes, animations, materials, scripts, prefabs, these are all concepts associated with Unity and these are also assets. Now, scenes is a type of asset.

Scenes are a specific type of assets. Assets that contain all or part of a game or application is assets. called a scene. You can create any number of scenes in a project, depends on how many layers levels are there. And Unity provides a scene interface to edit or create or modify a scene.



As shown on this screen, you can see this is a scene comprising of several components, each can be a different asset and so on. Another fundamental concept is game objects. So, these are the fundamental objects that represents characters, props or scenario. Now, game objects act as container for components which implement the functionality of any object. To give a game object the properties, you need to add components to it.

And through the editor interface, we can access lots of built-in component types so that we can assign it to game objects. We can also make our own components and game objects using scripting APIs. For instance, a light object can be built by adding the light component to a game object. with a script. Now, there are some primitive object types that can be created directly within Unity such as cube, sphere, capsule, cylinder, plane, basic shapes that are game objects.

Next is prefabs, another fundamental concept. It allows you to create, configure and store game objects, acting as a template to create new prefabs in the scene. We can think of prefabs as objects that the user can make instances of. For example, if you are dealing with a shooter game, then a certain type of bullet can be used as a prefab and then we can create prefabs. instances of such bullets multiple times around a level of a game.

So, the bullet type is a prefab. So, these are the basic concepts that we should know while dealing with Unity. So, let us summarize what we have covered so far. What have we learned? Got introduced to the basic idea of Unity where when we are using the term Unity, we are referring to both the game engine which is a middleware and the editor interface which allows the developer to create a system. So, we talked about the IDE which we get when we install the Unity Hub package, then the various components of the IDE and also some core concepts such as GameObject, Asset, Prefab and Scene. We are going to continue our discussion in the next part of this lecture on linking theory to practice. I hope you could follow the discussion. Of course, for more details on Unity, you have to refer to the developer's manual. Here, our focus is on the basic concepts and how we can relate theory to practice. So, we will not go into the details of all features of Unity. So, I hope you can understand the concepts and you can go through the documentation for further reading on your own. So, looking forward to meeting you in the next part of this lecture. Thank you and goodbye.

Happy learning.